

Commons Architecture — Deep Dive

Extended architectural analysis of the TypeScript Commons implementation

Scope: This expanded edition goes substantially beyond the short version and includes representative implementation code where code clarifies the architecture.

Core thesis: AI brings the intelligence; the Commons provides the table. The model is a participant and reasoning engine, not the authoritative store of Commons state.

1. Architectural Identity

The Commons is a governed state-and-context platform that can host AI participants. It is not merely a database, prompt, or AI wrapper. Its durable state exists outside any model conversation, allowing a session to end without destroying the system.

2. CommonsCore as the Kernel

CommonsCore is the coordinating center. It should orchestrate identity, validation, context, mutation, history, and persistence without absorbing every implementation detail. ImmutableTable owns records; ContextLayer owns working context; LMStudioClient owns model transport; persistence owns storage; governance owns authority.

3. ImmutableTable and Authoritative Records

ImmutableTable establishes an append-oriented record model. RecordEntry carries a record ID, data, and provenance. Immutability preserves historical evidence while allowing the Commons to evolve through additional records, status changes, interpretations, and superseding information.

4. Provenance

Current provenance identifies creator, timestamp, and table. This is the minimum traceability layer. A mature version can add operation ID, source, context, parent/superseded records, and authorization information. Authoritative provenance should be established by the Commons rather than trusted from model output.

5. Version History

Record provenance describes an individual record; version history describes Commons state at a point in time. The existing history concept should eventually be integrated into a single authoritative commit path so state-changing operations cannot bypass auditing.

6. ContextLayer

Context is the working environment for a task, not durable knowledge. It can contain selected records, observations, interpretations, constraints, and recent history. The current in-memory approach is useful for the prototype; the long-term requirement is that useful context be reconstructible after restart.

7. Persistence and Recovery

commons_state.json provides transparent prototype persistence. It is simple and inspectable, but not a complete transactional store. The architecture should hide the storage implementation behind an interface so JSON can later be replaced by SQLite or another transactional backend without rewriting the conceptual system.

8. Participants and Authority

Participant identity establishes who is acting. Authorization establishes what that actor may do. HUMAN and AI participant types are a foundation, not a complete permission system. AI access to a tool must not automatically grant authority to change authoritative Commons state.

9. LMStudioClient

LMStudioClient is a thin adapter to the local LM Studio OpenAI-compatible interface. This is a strength because the model provider can be replaced. A model response is output, not automatically an authoritative Commons mutation.

10. LocalToolkit and Operations

Tool capability and authority are separate concepts. A tool may technically perform an operation while the requesting participant is not authorized to apply it. Governance should sit above tool execution rather than being reinvented inside each tool.

11. Memory versus Working Context

Durable memory contains information intended to survive sessions. Working context contains material selected for the current task. The context should be assembled from durable Commons state, not become a substitute for that state.

12. Records, Observations, Interpretations, and Knowledge

The architecture benefits from separating what was recorded from what was noticed, interpreted, or derived. AI-generated interpretation can be produced quickly without silently becoming authoritative fact.

13. Novelty, Importance, Relevance, Confidence

These are distinct indexing dimensions. Novelty describes newness; importance describes consequence; relevance describes task usefulness; confidence describes support. They should not collapse into one status value. The planned lifecycle New, Emerging, Established, Dormant, Expired provides temporal state without erasing history.

14. Communication Review Layer

A contextual review layer between parsing and generation can select relevant information, identify novelty, distinguish authority from uncertainty, and assemble a sufficient working set. This is also a mechanism for preventing context bloat.

15. Layered Architecture

A practical responsibility stack is governance; identity and authority; authoritative records; provenance and history; context and interpretation; persistence; tools; and AI transport. These can remain within one application while retaining separate responsibilities.

16. Representative Record Code

The following excerpt shows the architectural meaning of giving records identity and provenance rather than storing anonymous objects.

```
interface Provenance {
  created_by: string;
  timestamp: number;
  table_id: string;
}

interface RecordEntry {
  record_id: string;
  data: any;
  provenance: Provenance;
}

class ImmutableTable {
  private _id: string;
  private _schema: string;
  public _data: RecordEntry[];

  constructor(tableId: string, schema: string) {
    this._id = tableId;
    this._schema = schema;
    this._data = [];
  }
}
```

```
}
```

17. Mutation and Commit Pipeline

A mature authoritative mutation can follow proposal -> validation -> authorization -> mutation -> provenance -> history -> persistence -> result. The exact implementation can evolve, but a consistent commit boundary prevents bypass paths.

```
async function commitOperation(operation: Operation) {
  const actor = participants.require(operation.actorId);
  authorization.require(actor, operation);

  const proposed = await validate(operation);
  const result = applyToWorkingState(proposed);

  recordProvenance(result, makeProvenance(actor, operation));

  const version = history.commit({
    operationId: operation.id,
    actor: actor.name,
    state: snapshot()
  });

  await persistence.writeAtomic(snapshot());
  return { operationId: operation.id,
    versionId: version.id, result };
}
```

Design skeleton, not a claim that this exact function already exists in the prototype.

18. Failure and Recovery

Model timeouts, invalid tool output, interrupted writes, and process termination must be expected. A dead AI session must not mean a dead Commons. Persistence should preserve the last valid state and eventually support stronger journaling or transactions.

19. Security and Governance

Participant type is not a security system. Production use requires authentication, authorization, capability checks, and auditable operations. The important boundary is preventing generated AI output from becoming authority merely because a tool is available.

20. Scaling Beyond JSON

JSON is appropriate for an inspectable prototype. As the Commons grows, concurrent writes, indexing, transactions, and queries favor a transactional backend. A persistence interface allows that migration without changing higher-level architecture.

21. Testing the Architecture

Tests should verify invariants: authoritative records have provenance; unauthorized actors are rejected; invalid patches fail; saved state restores correctly; mutations create history; and a fresh AI session can reconstruct required context.

22. What the Prototype Already Establishes

The implementation already contains a meaningful architectural spine: immutable records, provenance, participant identity, context, history, persistence, local operations, centralized coordination, and replaceable AI transport.

23. What Remains to Be Built

Major remaining work is enforcement and integration: automatic history commits, reconstructible contexts, stronger schemas, authorization, transaction-safe persistence, richer provenance, retrieval/indexing, migration, and governance enforcement.

24. Engineering Sequence

A foundation-first sequence is: reliable persistence; one authoritative mutation path; automatic provenance; automatic history; reconstructible context; authorization; schema enforcement; retrieval/indexing; governance; and hardened AI session reconstruction.

25. Central Architectural Principle

AI brings the intelligence; the Commons provides the table. The AI is not the database, prompt, or permanent memory. The Commons is the durable, governed environment from which the AI receives a working view and to which accepted changes are committed.

26. Final Assessment

The TypeScript implementation contains the beginnings of a real Commons kernel. Its major components are arranged around session independence: Leo should be able to stop, restart, change conversations, or eventually change models without losing durable Commons state. The next phase is primarily about enforcing the boundaries already present.

Appendix — Architectural Invariants

- Every authoritative record has system-established provenance.
- AI output is not automatically authoritative.
- Durable state survives model sessions.
- Authoritative mutations are auditable.
- Identity is distinct from authorization.
- Working context can be reconstructed.
- Confidence remains distinct from status, importance, relevance, and novelty.

Appendix — Session Reconstruction

```
async function startSession(request: SessionRequest) {  
  const state = await persistence.load();  
  const actor = participants.require(request.actorId);  
  
  const records = retrieval.select(state, request.task);  
  
  const context = contextBuilder.build({  
    actor,  
    task: request.task,  
    records,  
    rules: rules.activeFor(actor),  
    recentHistory: history.relevant(state)  
  });  
  
  return ai.createWorkingSession(context);  
}
```

The important property is that the working session is reconstructed from Commons state. The model conversation is a workspace, not the permanent store.

End of extended deep dive.